



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Licenciatura en Sistemas de Información

Base de Datos 2

Grupo N°10

Integrantes:

García, Edgar W.	U: 48446
López Jorge Daniel	U: 52778
Mac Lean Juan Manuel	U: 47473
Rodríguez, Jorge O.	U: 47219
Villalba, Ricardo D.	U: 43501

Monografía: CONCEPTOS PARA BASES DE DATOS ORIENTADAS A OBJETOS

2020

Índice

1. Introducción	5
2. Conceptos generales sobre la orientación a objetos.	6
2.1. Estructura General de un Objeto.	7
2.1.1. OID – Identidad de un Objeto	7
2.1.2. La Triada (I, C, V).....	8
2.1.3. Representación Gráfica de un Objeto	9
2.1.4. Igualdad entre Objetos	10
2.1.5. Constructores de Tipo	11
3. Encapsulamiento de Operaciones, Métodos y Persistencia.	13
3.1. Persistencia de Objetos a través del Nombramiento y la Alcanzabilidad.....	15
4. Jerarquías de Tipo y Herencia	17
4.1. Restricciones en las Extensiones de una Jerarquía de Tipos	18
4.2. Herencia Múltiple y Herencia Selectiva.....	18
4.3. Objetos Complejos.....	19
4.3.1. Objetos Complejos No Estructurados	19
4.3.2. Objetos Complejos Estructurados	20
5. Conclusión	22
6. Referencia Bibliografía	23

Índice de Figuras

Ilustración 1: “Representación Gráfica de un Objeto” [1].....	10
Ilustración 2: “Interacción de Objetos Encapsulados” [2].....	13

Índice de algoritmos

Ecuación 1: “Algoritmo Constructor” [1]	11
Ecuación 2: “Algoritmos Empleado y Departamento” [1]	15
Ecuación 3: “Persistencia por Nombramiento” [1]	16

1. Introducción

Los sistemas y modelos de datos tradicionales, como el relacional, el jerárquico y el modelo de red, han sido bastante exitosos en el desarrollo de bases de datos para aplicaciones tradicionales. Sin embargo, muestran ciertas deficiencias cuando hablamos de aplicaciones para bases de datos más complejas (por ejemplo, bases de datos para diseños de ingeniería, experimentos científicos, telecomunicaciones, sistemas de información geográfica y multimedia). Estas nuevas aplicaciones poseen requerimientos y características que difieren de las de las aplicaciones tradicionales, como estructuras más complejas para los objetos, transacciones de mayor duración, nuevos tipos de datos para almacenar imágenes o ítems textuales de tamaño considerable y la necesidad de definir operaciones específicas para la aplicación que no pertenecen a ningún estándar. Las **bases de datos orientas a objetos** pretenden cubrir las necesidades de estas aplicaciones más complejas. La aproximación de la orientación a objetos, ofrece la flexibilidad para manejar estos requerimientos sin estar limitados por los tipos de datos y lenguajes de consulta disponibles en los sistemas de bases de datos tradicionales. Una característica clave de las BDOO es el poder que le dan al diseñador para especificar tanto la estructura de los objetos complejos como las operaciones que pueden ser aplicadas a dichos objetos.

La necesidad de características adicionales para el modelado de datos también ha sido percibida por los fabricantes de sistemas de gestión de bases de datos, y sus sistemas más recientes incluyen muchas de las características propuestas para las bases de datos orientadas a objetos. Esto ha llevado a los sistemas a ser conocidos como SGBD objeto-relacionales o SGBD relacionales extendidas.

A medida que iban apareciendo SGBD orientadas a objetos, se descubrió la necesidad de un estándar del lenguaje y el modelo. Dado que el procedimiento formal para aprobar un estándar lleva unos años, un consorcio de vendedores y usuarios de SGBD orientadas a objetos, llamado ODMG (Object Database Management Group) propusieron un estándar conocido como ODMG-93, que ha sido revisado dando lugar a la última versión, conocida como ODMG versión 2.0.

2. Conceptos generales sobre la orientación a objetos.

Los lenguajes de programación OO tienen sus raíces en el lenguaje SIMULA, propuesto en el año 1960. En dicho lenguaje, el concepto de clase agrupa la estructura interna de un objeto en una declaración de clase. En consecuencia, los investigadores propusieron el concepto de tipo de datos abstracto, que ocultaba la estructura de datos interna y especificaba todas las operaciones externas que se le podían aplicar a dicha estructura, llegando al concepto de encapsulación.

Un objeto consta de dos componentes: el estado (el valor) y el comportamiento (las operaciones). Por lo tanto, es algo similar a una variable de programa en un lenguaje de programación, excepto que típicamente tendrá una compleja estructura de datos, así como unas operaciones específicas definidas por el programador. Los objetos en lenguajes de programación orientados a objetos existen únicamente durante la ejecución del programa y se les llama objetos transitorios. Una base de datos OO puede alargar la existencia de los objetos de forma que sean almacenados de forma permanente, y así los objetos persisten incluso después de la ejecución del programa y puedan ser recuperados más tarde y compartidos con otros programas. En otras palabras, las bases de datos OO almacenan objetos persistentes permanentemente en un almacenamiento secundario, y permite la compartición de esos objetos con otros programas y aplicaciones. Esto requiere la incorporación de otros detalles relacionados con bases de datos, como son mecanismos de indexación, control de concurrencia y recuperación.

La estructura interna de un objeto en lenguajes de programación OO incluye la especificación de variables de instancias, que almacenan el valor que define el estado interno del objeto. De esta forma, una variable de instancia es similar al concepto de un atributo, excepto que las variables de instancia pueden ser encapsuladas en los objetos y así no es necesario que sean visibles a usuarios externos, así como también pueden ser de tipos de datos arbitrariamente complejos. Además, los sistemas OO permiten la definición de operaciones o funciones que pueden ser aplicadas a objetos de un tipo en particular. De hecho, algunos modelos de OO insisten en que todas las operaciones que un usuario pueda aplicar a un objeto deben ser predefinidas. A la hora de encapsular estas operaciones, la definición se realiza en dos partes, en la primera, llamada la signatura o interfaz de la operación, se especifican el nombre de la operación y los argumentos. En la segunda parte, llamada el método o el cuerpo, se especifica la implementación de la operación. Estas operaciones pueden ser invocadas mediante el paso de un mensaje al objeto, que incluye el nombre de la operación y los argumentos.

Entonces, el objeto ejecuta el método para dicha operación. Esta encapsulación permite la modificación, de la estructura interna de un objeto, así como la implementación de sus operaciones sin tener que molestar a programas externos que invocan esas operaciones. De esta forma, la encapsulación proporciona cierta independencia de los datos y las operaciones.

Otros conceptos clave en los sistemas OO son la jerarquía de tipos y clases y la herencia. Esto permite la especificación de nuevos tipos de clases que heredan parte de la estructura y operaciones de otros tipos que fueron previamente definidos. Esto facilita el desarrollo incremental de tipos de datos de un sistema, y la reutilización de definiciones de tipos existentes cuando creamos nuevos tipos de objetos.

El polimorfismo es otra de las características de los sistemas OO, es la capacidad de que una operación pueda ser aplicada a diferentes tipos de objetos. En una situación así, un nombre de operación puede referirse a distintas implementaciones, dependiendo del tipo de objeto al que se le está realizando la operación. Esta característica también se denomina “operator overloading” o sobrecarga de operadores.

2.1. Estructura General de un Objeto.

2.1.1. OID – Identidad de un Objeto

Un sistema de bases de datos orientado a objetos proporciona una identidad única a cada objeto independiente almacenado en la base de datos. Esta identidad, normalmente se implementa mediante un identificador de objeto único (OID), generado por el sistema. El valor de este identificador no es visible al usuario externo, pero se usa internamente por el sistema para identificar a cada objeto y poder crear y manejar referencias entre objetos.

La principal propiedad de un OID es que debe ser inmutable; es decir, el valor del OID de un determinado objeto no debe cambiar nunca. Esto preserva la identidad del objeto del mundo real que se representa. Por lo tanto, una BDOO debe tener algún mecanismo para generar identificadores únicos y que permita preservar la propiedad de inmutabilidad de los mismos. También sería deseable que cada identificador se usara una sola vez, o sea, que incluso si un objeto se elimina de la BD, su identificador no sea asignado a otro objeto. Estas propiedades implican que el OID no depende de ninguno de los atributos del objeto, ya que éstos pueden cambiarse o corregirse. También se considera como inapropiado basar el OID del objeto en la dirección física del objeto almacenado, porque la misma dirección puede cambiar cuando se efectúe una

reorganización física de la BD. Sin embargo, algunos sistemas utilizan la dirección física para aumentar la eficiencia al obtener un objeto. Si la dirección cambiase, se utilizaría un apuntador indirecto, que daría la nueva dirección.

2.1.2. La Triada (I, C, V)

Una forma de representar los objetos de una BDOO es ver a cada uno como un trío (i, c, v), donde “i” es el identificador de objetos (OID) único, “c” es un constructor de tipo y “v” es el estado del objeto. En las BDOO, el estado (valor actual) de un objeto complejo puede construirse a partir de otros objetos utilizando ciertos constructores de tipos.

Los constructores “c” más utilizados son los de átomo, tupla y conjunto, junto a los de lista, bolsa y array. A partir de ellos se interpreta el estado de cada objeto (“v” en el trío). Si “c” es átomo, entonces “v” será algún valor atómico perteneciente al dominio de valores básicos que utiliza el sistema. Si “c” es conjunto, “v” es un conjunto de identificadores de objetos ({i1, i2, i3...}). Con “c” siendo una tupla, “v” es una tupla de la forma <a1:i1, a2:i2, a3:i3...> donde aj es un nombre de atributo, y cada ij es un identificador de objeto. Con “c” siendo lista “v” es una lista ordenada [i1, i2, i3...] de identificadores de objetos del mismo tipo.

La lista es similar al conjunto salvo en que la lista está ordenada, por lo que podemos hablar del i-ésimo elemento. Una lista puede tener un número arbitrario de elementos. Pero si hablamos de un número determinado, entonces “c” es un array.

Una bolsa se diferencia del conjunto en que en el conjunto todos los elementos deben ser diferentes, mientras que en la bolsa puede haber duplicados.

Ejemplo de objeto complejo:

```
o1 = (i1, ÁTOMO, 'HOUSTON')
o2 = (i2, ÁTOMO, 'BELLAIRE')
o3 = (i3, ÁTOMO, 'SUGARLAND')
o4 = (i4, ÁTOMO, 5)
o5 = (i5, ÁTOMO, 'INVESTIGACIÓN')
o6 = (i6, ÁTOMO, '22-05-1988')
o7 = (i7, CONJUNTO, {i1, i2, i3})
o8 = (i8, TUPLA, <NOMBRED:i5, NUMEROD:i4, JF:i9, LOCALIZACIONES:i7,
EMPLEADOS:i10, PROYECTOS:i11>)
o9 = (i9, TUPLA, <JEFE:i12, FECHA_INIC_JEFE:i6>)
o10 = (i10, CONJUNTO, {i12, i13, i14})
o11 = (i11, CONJUNTO, {i15, i16, i17})
```


o12 = (i12, TUPLA, <NOMBRE:i18, INIC:i19, APELLIDO:i20, NSS:i21, SALARIO:i26, SUPERVISOR:i27, DEPT:i8>)

Puede verse que los primeros seis objetos son simplemente valores atómicos. Pero hay otros que están compuestos por otros objetos. El objeto o7 es un objeto con valor de conjunto que representa todas las ubicaciones de un departamento. El objeto o8 es un objeto con valor de tupla que representa al departamento 5. El objeto o9 representa a un jefe de departamento, y el objeto o12 representa a un empleado.

2.1.3. Representación Gráfica de un Objeto

En este modelo, un objeto se puede representar como una estructura de grafo que puede construirse aplicando recursivamente los constructores de tipos. El grafo que representa a un objeto “oi” puede construirse creando primero un nodo para el propio objeto “oi”. El nodo para “oi” se etiqueta con el OID y el constructor del objeto “c”. También creamos un nodo en el grafo para cada valor atómico básico. Si un objeto “oi” tiene un valor atómico, dibujamos un arco tendente desde el nodo que representa “oi” hasta el nodo que representa su valor básico. Si construimos el valor del objeto, dibujamos arcos tendentes desde el nodo del objeto hasta el nodo que representa el valor construido.

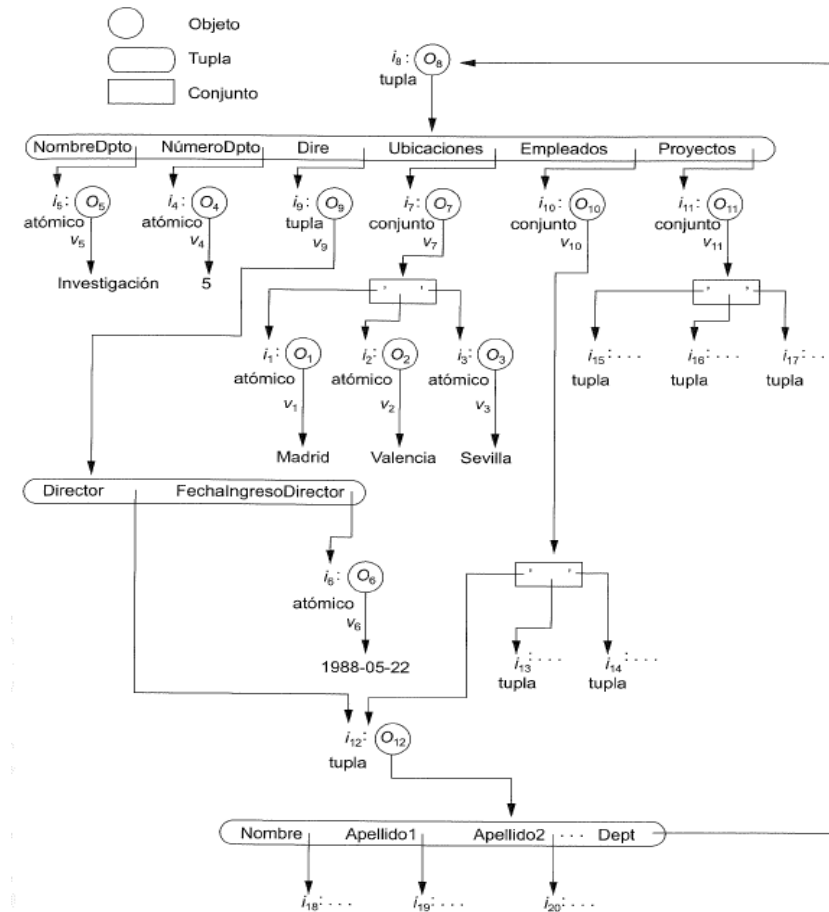


Ilustración 1: "Representación Gráfica de un Objeto" [1]

2.1.4. Igualdad entre Objetos

El modelo anterior permite dos tipos de definiciones en una comparación por igualdad de los estados de dos objetos. Se dice que dos objetos tienen estados idénticos (igualdad profunda) si los gráficos que representan sus estados son idénticos en cada respecto, incluyendo los OIDs de cada nivel. Por otro lado, la definición débil de igualdad es cuando dos objetos tienen estados iguales (igualdad poco profunda o superficial). En este caso, las estructuras gráficas deben ser iguales, y todos los valores atómicos correspondientes de los gráficos también deben ser iguales. Sin embargo, algunos nodos internos correspondientes en los dos gráficos pueden tener objetos con OIDs diferentes.

Ejemplo de objetos idénticos y objetos iguales.

o1 = (i1, TUPLA, <A1:i4, A2:i6>)

o2 = (i2, TUPLA, <A1:i5, A2:i6>)

o3 = (i3, TUPLA, <A1:i4, A2:i6>)

o4 = (i4, ÁTOMO, 10)

o5 = (i5, ÁTOMO, 10)

o6 = (i6, ÁTOMO, 20)

Los objetos “o1” y “o2” tienen estados de igualdad, puesto que sus estados a nivel atómico son iguales, pero los valores se alcanzan a través de los objetos “o4” y “o5” distintos. No obstante, los estados de los objetos “o1” y “o3” son idénticos, aunque los propios objetos no lo son porque tienen OIDs distintos. De forma parecida, aunque los estados de “o4” y “o5” son idénticos, los objetos reales “o4” y “o5” son iguales pero no idénticos, porque tienen OIDs distintos.

2.1.5. Constructores de Tipo

Se puede utilizar un lenguaje de definición de objetos (ODL) para definir los tipos de objetos para una aplicación de base de datos determinada. Los constructores de tipos pueden servirnos para definir las estructuras de datos de un esquema de base de datos orientada a objetos.

Ejemplo:

```

1  DEFINE TYPE EMPLEADO
2  TUPLE (      NOMBRE:      STRING;
3              INIC:        CHAR;
4              APELLIDO:    STRING;
5              NSS:         STRING;
6              FECHANACIMIENTO: FECHA;
7              DIRECCIÓN:  STRING;
8              SEXO:        CHAR;
9              SALARIO:     FLOAT;
10             SUPERVISOR:  EMPLEADO;
11             DEPT:        DEPARTAMENTO; );
12  DEFINE TYPE FECHA
13  TUPLE (      AÑO:        INTEGER;
14              MES:        INTEGER;
15              DÍA:        INTEGER; );
16
17  DEFINE TYPE DEPARTAMENTO
18  TUPLE (      NOMBRED:    STRING;
19              NÚMEROD:    INTEGER;
20              JF:        TUPLE (      JEFE:      EMPLEADO;
21                                  FECHAINICIO: FECHA; );
22              LOCALIZACIONES: SET (STRING);
23              EMPLEADOS:   SET (EMPLEADO);
24              PROYECTOS:   SET (PROYECTO); );

```

Ecuación 1: “Algoritmo Constructor” [1]

Los atributos que hacen referencia a otros objetos pueden considerarse como referencias a otros objetos, y por lo tanto pueden usarse para representar relaciones entre objetos. El valor del atributo en cuestión podría ser el OID del objeto al que referencia. Una relación binaria puede representarse en una dirección, o bien tener una referencia inversa. Por ejemplo, el objeto Empleado tiene una referencia al departamento en el que trabaja, y el objeto Departamento puede tener una serie de referencias a empleados que trabajan en él. El estándar ODMG 2.0 permite declarar explícitamente los inversos como atributos de relación para asegurar que las referencias inversas sean consistentes.

3. Encapsulamiento de Operaciones, Métodos y Persistencia.

El concepto de encapsulación es una de las principales características de los lenguajes y sistemas OO. Está relacionado con los conceptos de tipos de datos abstractos y de ocultación de la información en los lenguajes de programación. En los sistemas tradicionales no era lo habitual, dado que la costumbre era hacer que la estructura de los objetos fuera visible para los usuarios y los programas externos.

La idea principal es definir el comportamiento de un tipo de objeto basado en las operaciones que se pueden aplicar externamente a objetos de ese tipo. La estructura interna del objeto queda oculta y sólo se puede acceder a ella mediante una serie de operaciones predefinidas (para crear y destruir los objetos, para modificar su estado, o para obtener información de él). En general, la implementación de una operación puede especificarse en un lenguaje de programación de propósito general que ofrezca flexibilidad y potencia para definir las operaciones.

Mediante esta idea, los usuarios sólo perciben la interfaz del objeto donde se definen los nombres y argumentos de las operaciones que pueden hacerse con él. La implementación queda oculta, que incluye la definición de las estructuras de datos y la implementación de las operaciones que permiten acceder a esas estructuras.

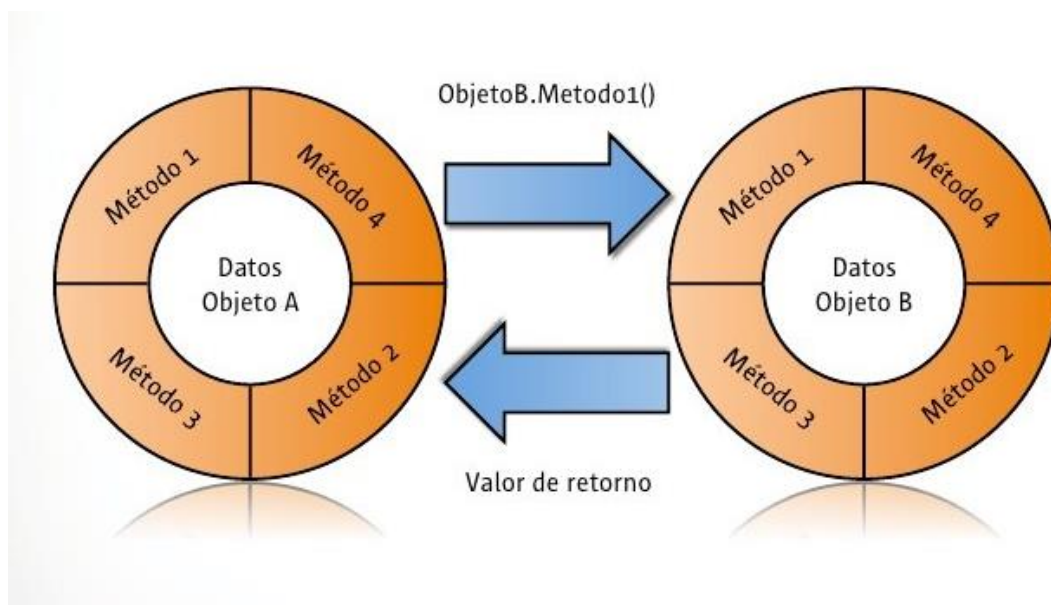


Ilustración 2: "Interacción de Objetos Encapsulados" [2]

En las aplicaciones de BD, el requisito de que todos los objetos queden encapsulados es demasiado estricto. Una forma de relajarlo un poco consiste en dividir

la estructura en atributos visibles y ocultos. Los operadores externos o un lenguaje de consulta de alto nivel pueden tener acceso directo a los atributos visibles para leerlos. Casi todos los SGBGOO emplean lenguajes de consulta de alto nivel para tener acceso a los atributos visibles. El lenguaje de consulta OQL se propone como lenguaje de consulta estándar para BDOO.

El término clase se utiliza para referirse a una definición de tipo de objetos, junto con las definiciones de las operaciones para ese tipo. Para cada clase se declaran varias operaciones, y la signatura de cada operación se incluye en la definición de la clase. Para cada operación se debe definir un método (implementación) en otro lugar. Las operaciones más habituales incluyen el constructor de objeto, el destructor y varias operaciones para modificar el objeto. Otras operaciones nos permitirán recuperar información acerca del objeto.

Ejemplo definición de clase “Empleado” y “Departamento”

```

1      DEFINE CLASS EMPLEADO
2      TYPE TUPLE (  NOMBRE:          STRING;
3                    INIC:            CHAR;
4                    APELLIDO:        STRING;
5                    FECHANACIMIENTO: FECHA;
6                    DIRECCIÓN:      STRING;
7                    SEXO:            CHAR;
8                    SALARIO:         FLOAT;
9                    SUPERVISOR:      EMPLEADO;
10                   DEPT:            DEPARTAMENTO;  );
11                   OPERATIONS  EDAD:            INTEGER;
12                   CREAR_EMP:    EMPLEADO;
13                   BORRAR_EMP:   BOOLEAN;
14      END EMPLEADO;

15     DEFINE CLASS DEPARTAMENTO
16     TYPE TUPLE (  NOMBRED:          STRING;
17                   NÚMEROD:         INTEGER;

```

```

18                                JF:          TUPLE ( JEFE:          EMPLEADO;
19                                FECHAINICIO:  FECHA;          );
20                                LOCALIZACIONES:SET (STRING);
21                                EMPLEADOS:    SET (EMPLEADO);
22                                PROYECTOS:    SET (PROYECTO);  );
23                                OPERATIONS    NUM_DE_EMPS: INTEGER;
24                                CREAR_DEPT:   DEPARTAMENTO;
25                                BORRAR_DEPT:   BOOLEAN;
26                                ASIGNAR_EMP(E: EMPLEADO):   BOOLEAN;
27                                (* AÑADE UN EMPLEADO AL DEPARTAMENTO *)
28                                QUITAR_EMP(E: EMPLEADO):   BOLEAN;
29                                (* ELIMINA UN EMPLEADO DEL DEPARTAMENTO *)
30                                END DEPARTAMENTO;

```

Ecuación 2: "Algoritmos Empleado y Departamento" [1]

Una operación se aplica automáticamente a un objeto mediante la notación de punto. Por ejemplo, para averiguar el número de empleados de un departamento d, haríamos d.num_de_emps.

3.1. Persistencia de Objetos a través del Nombramiento y la Alcanzabilidad

Lo normal es que los objetos sean creados por programas en ejecución, al invocar la operación de construcción de objetos. Por supuesto, no todos los objetos están destinados a ser almacenados en la BD. Hay objetos transitorios que sólo existen durante la ejecución del programa. Los objetos persistentes, por su parte, son los que se almacenan en la BD y persisten después de la ejecución del programa. Los mecanismos habituales para hacer persistente un objeto son el nombramiento y la alcanzabilidad.

El mecanismo de nombramiento implica dar al objeto un nombre persistente y único mediante el cual pueda ser recuperado. Ese nombre persistente se le puede dar mediante una sentencia o una operación en un programa. Todos los nombres que se den a los objetos deben ser únicos en cada base de datos particular. Así, los objetos persistentes nombrados se usan como puntos de entrada a la base de datos. Obviamente,

en una BD extensa, con miles de objetos no resulta nada práctico dar nombres a todos los objetos, por lo que a la mayoría de los objetos se los convierte en persistentes utilizando el segundo método, la alcanzabilidad. Éste funciona haciendo que el objeto sea alcanzable desde algún objeto persistente. Se dice que un objeto B es alcanzable desde un objeto A si una secuencia de referencias del gráfico del objeto conduce del objeto A al objeto B.

Ejemplo de persistencia por nombramiento y alcanzabilidad.

```

1      DEFINE CLASS CONJUNTODEPARTAMENTO
2          TYPE          SET (DEPARTAMENTO);
3          OPERATIONS    AÑADIR_DEPT (D: DEPARTAMENTO): BOOLEAN;
4          (* AÑADE UN DEPARTAMENTO AL OBJETO CONJUNTODEPARTAMENTO *)
5                      QUITAR_DEPT (D: DEPARTAMENTO): BOOLEAN;
6          (* ELIMINA UN DEPARTAMENTO DEL OBJETO CONJUNTODEPARTAMENTO *)
7                      CREAR_CONJUNTO_DEPT: CONJUNTODEPARTAMENTO;
8                      BORRAR_CONJUNTO_DEPT:      BOOLEAN;
9      END CONJUNTODEPARTAMENTO;
10     .....
11     PERSISTENT NAME TODOSDEPARTAMENTOS: CONJUNTODEPARTAMENTO;
12     (* TODOSDEPARTAMENTOS ES UN OBJETO NOMBRADO PERSISTENTE DE TIPO
13     CONJUNTODEPARTAMENTO *)

```

Ecuación 3: "Persistencia por Nombramiento" [1]

Hemos creado un objeto del tipo ConjuntoDepartamento, y le hemos dado un nombre, haciéndolo así persistente. Cualquier objeto del tipo Departamento que se añada al conjunto con la operación pertinente se vuelve persistente en virtud de ser alcanzable desde TodosDepartamentos. Al objeto TodosDepartamentos se le denomina a menudo la extensión de la clase Departamento, ya que contendrá todos los objetos persistentes de tipo Departamento. El estándar ODL de ODMG ofrece al diseñador del esquema la opción de nombrar una extensión como parte de la definición de la clase.

En el enfoque OO, una declaración de clase sólo especifica el tipo y las operaciones de una clase de objetos. El usuario debe definir por separado un objeto persistente de tipo conjunto o lista cuyo valor sea la colección de referencias a todos los objetos persistentes.

4. Jerarquías de Tipo y Herencia

Dado que en gran parte de aplicaciones de BD hay un gran número de objetos, sería deseable poderlos clasificar según su tipo. Pero además, tenemos un requisito adicional: que el sistema permita la definición de nuevos tipos basados en tipos predefinidos, dando origen a una jerarquía de tipos (o de clases).

Por lo general, para definir un tipo se le asigna un nombre de tipo y una serie de atributos y operaciones para el mismo. En algunos casos, los atributos y las operaciones se denominan funciones, porque podemos considerar a los atributos como operaciones con cero argumentos.

NOMBRE_DE_TIPO: FUNCIÓN, FUNCIÓN,..., FUNCIÓN

PERSONA: NOMBRE, DIRECCIÓN, FECHANAC, EDAD, NSS

En el tipo PERSONA, el nombre, la dirección la fecha de nacimiento y el NSS pueden implementarse como atributos almacenados, mientras que la edad se puede implementar mediante un método que calcula la edad a partir del valor del atributo Fechanac y de la fecha actual.

Cuando el diseñador o el usuario desean crear un nuevo tipo similar, pero no idéntico, a un tipo ya existente, resulta útil el concepto de subtipo. El subtipo heredaría todas las funciones del tipo predefinido, al cual se le llama supertipo.

EMPLEADO: NOMBRE, DIRECCIÓN, FECHANAC, EDAD, NSS, SALARIO, FECHAALTA, ANTIGÜEDAD

ESTUDIANTE: NOMBRE, DIRECCIÓN, FECHANAC, EDAD, NSS, TITULACIÓN, NM

Como ESTUDIANTE y EMPLEADO incluyen todas las funciones de PERSONA más algunas funciones adicionales, podemos declararlos como subtipos de PERSONA. Los dos heredarán las funciones definidas en PERSONA. Así, sólo será necesario definir las nuevas funciones que no se heredan.

La idea de definir un tipo implica definir todas sus funciones e implementarlas. Cuando se define un subtipo, puede heredar todas estas funciones y su implementación. Sólo es necesario definir e implementar las funciones que son específicas o locales para el subtipo y que no están implementadas en el supertipo.

EMPLEADO SUBTYPE-OF PERSONA: SALARIO, FECHAALTA, ANTIGÜEDAD

ESTUDIANTE SUBTYPE-OF PERSONA: TITULACIÓN, NM

En general, un subtipo incluye todas las funciones que están definidas para su supertipo, más algunas adicionales que le son específicas. Por ello, es posible definir una

jerarquía de tipos para indicar los vínculos subtipo/supertipo entre todos los tipos declarados en el sistema.

4.1. [Restricciones en las Extensiones de una Jerarquía de Tipos](#)

En las aplicaciones de BD es habitual que cada tipo o subtipo tenga una extensión asociada a él, que contenga la colección de todos los objetos persistentes de ese tipo o subtipo. En ese caso, la restricción es que todos los objetos de una extensión que pertenezcan a un subtipo deben ser también miembros de la extensión que corresponda a su supertipo. En algunos sistemas, existe un tipo predefinido (llamado RAIZ o clase OBJETO), cuya extensión contiene a todos los objetos del sistema. Entonces, se lleva a cabo una clasificación asignando los objetos a subtipos adicionales, creando una jerarquía de tipos o una jerarquía de clases para el sistema. En el modelo ODMG el usuario puede especificar una extensión para cada tipo dependiendo de la aplicación.

En la mayor parte de los sistemas OO, se hace una distinción entre objetos y colecciones persistentes y transitorias. Una colección persistente contiene una colección de objetos que se almacena permanentemente en la BD, de modo que múltiples programas puedan tener acceso a ella. Una colección transitoria sólo existe temporalmente durante la ejecución de un programa, que desaparece cuando el programa termina.

4.2. [Herencia Múltiple y Herencia Selectiva](#)

La herencia múltiple es una jerarquía de tipos que se da cuando un cierto subtipo es un subtipo de dos (o más) tipos diferentes y por tanto hereda las funciones de ambos supertipos. Un problema que puede presentarse con la herencia múltiple se da cuando los dos supertipos pueden tener funciones distintas con el mismo nombre, lo que da lugar a la ambigüedad. La regla general es que, si una función se hereda de algún supertipo común, entonces será heredada una sola vez. Así nos evitamos la ambigüedad; el problema sólo surge cuando las funciones son distintas en los dos supertipos.

Hay varias técnicas para resolver la ambigüedad en la herencia múltiple. Una solución es que el sistema compruebe si hay ambigüedad en el momento de crearse el subtipo, y dejar que el usuario decida qué función se heredará. Otra solución es utilizar una opción por defecto del sistema. Otra sería prohibir la herencia múltiple si hay

ambigüedad de nombre, obligando al usuario a cambiar el nombre de una de las funciones.

La herencia selectiva se produce cuando un subtipo hereda sólo algunas funciones de un supertipo. Puede usarse una cláusula EXCEPT para enumerar las funciones que el subtipo no debe heredar. No es usual que los sistemas de BDOO cuenten con este mecanismo.

4.3. Objetos Complejos

Una de las motivaciones para el desarrollo de sistemas OO fue el deseo de representar objetos complejos. Hay objetos complejos de dos tipos: estructurados y no estructurados. Un objeto complejo estructurado está constituido por componentes y se define aplicando los constructores de tipo disponibles recurrentemente en diversos niveles. Uno no estructurado casi siempre es un tipo de datos que requiere una gran cantidad de almacenamiento, como por ejemplo un tipo de datos que representa una imagen.

4.3.1. Objetos Complejos No Estructurados

Un objeto complejo no estructurado permite el almacenamiento y recuperación de objetos de gran tamaño. Son ejemplos representativos las imágenes de mapas de bits y las cadenas de texto largas. También se conocen como objetos binarios extensos, o BLOB (binary large object). Estos objetos son no estructurados en el sentido de que el SGBD no sabe qué estructura tienen. Sólo la aplicación que los usa sabe cómo interpretarlos. Se consideran complejos porque necesitan un área de almacenamiento sustancial y no forman parte de los tipos de datos estándar que suelen ofrecer los SGBD. Como el tamaño de los objetos es considerable, un SGBD podría obtener una porción del objeto y proporcionarla al programa antes de obtener todo el objeto. También podría usar técnicas de búferes y de almacenamiento en memoria cache para obtener por anticipado porciones del objeto, antes de que el programa las necesite.

El software de SGBD no tiene capacidad de procesar directamente las condiciones de selección ni otras basadas en valores de estos objetos, a menos que la aplicación proporcione el código necesario para realizar las operaciones de comparación (en el caso de la selección). En un SGBDOO, esto se puede conseguir definiendo un nuevo tipo de

datos abstracto para los objetos no interpretados y suministrando los métodos para seleccionar, comparar y mostrar esos objetos.

Como un SGBDOO permite a los usuarios crear nuevos tipos, y como un tipo incluye tanto estructura como operaciones, podemos considerar que un SGBDOO tiene un sistema de tipos extensibles. Se pueden crear bibliotecas de nuevos tipos definiendo estructura y operaciones. Otras aplicaciones pueden utilizar estos tipos, e incluso modificarlos (creando subtipos de los tipos provistos en las bibliotecas).

4.3.2. Objetos Complejos Estructurados

La diferencia entre objeto complejo no estructurado y objeto complejo estructurado consiste en que la estructura del objeto se define por la aplicación repetida de los constructores de tipos proporcionados por el SGBDOO. Así, la estructura está definida, y además, el SGBDOO la conoce.

Existen dos tipos de semántica para la referencia entre un objeto complejo y sus componentes en cada nivel. El primer tipo, que llamamos semántica de propiedad, se aplica cuando los subobjetos de un objeto complejo están encapsulados dentro de éste, y por ello se los considera parte de él. El segundo tipo, llamado semántica de referencia, se aplica cuando los componentes del objeto complejo son objetos independientes, pero pueden ser referenciados desde el objeto complejo. El primer tipo también es conocido como la relación es-parte-de o componente-de; y el segundo tipo como está-asociado-a. La relación es-parte-de tiene la característica de que los objetos están encapsulados dentro del objeto complejo y se consideran parte del estado interno del objeto. No necesitan identificadores del objeto y sólo acceden a ellos los métodos de ese objeto. Además, desaparecen junto con el objeto complejo al eliminarlo. Por otra parte, un objeto complejo con componentes referenciados se considera formado por objetos independientes con identidad y métodos propios. Cuando un objeto complejo necesita acceder a los componentes referenciados, debe hacerlo utilizando los métodos apropiados de los componentes. Además, como más de un objeto complejo puede hacer referencia a un mismo componente, este último no desaparece cuando se elimina el objeto complejo.

Los SGBDOO deben ofrecer opciones de almacenamiento para agrupar los objetos componentes de un objeto complejo en el almacenamiento secundario, para aumentar la eficiencia de las operaciones que accedan a ellos. En muchos casos la estructura del objeto se almacena sin ser interpretado en páginas del disco. Cuando se

recupera una página que incluye un objeto, el SGBDOO puede construir el objeto complejo a partir de la información contenida en las páginas del disco, las cuales pueden hacer referencia a páginas de disco adicionales. Eso se conoce como ensamblaje de objetos complejos.

5. Conclusión

Como conclusión podemos decir que las bases de datos orientadas a objetos es una de las opciones cada vez más utilizada para el análisis, desarrollo o mantenimiento de las aplicaciones actuales que manejen grandes volúmenes de información y tipos de datos.

Estas bases de datos permiten aplicar el modelo conceptual orientado a objetos tan conocido con cada una de sus características (abstracción, encapsulación, modularidad, jerarquía, tipos, concurrencia, herencia, polimorfismo, etc.) y así reducir los problemas del operador y hacer más eficiente el manejo de tipos de datos complejos generando un mayor rendimiento al compararlas con las bases de datos relacionales.

Sin embargo, estos dos enfoques (el relacional y el orientado a objeto) también brindan hoy en día la posibilidad de ser aplicados conjuntamente, expresando de esta manera las ventajas que puede brindar cada una de ellas.

6. Referencia Bibliografía

- [1] R. Elmasri and S. B. Navathe. “Fundamentos de sistemas de Base de Datos”, 5th ed. Pearson: London, UK, 17 Marzo 2006.
- [2] Slideshare.net “Conceptos Basicos de Programacion Orientada a Objetos” Mayo 2017. [Online], Disponible:
<https://www.slideshare.net/MariaGarcia38/conceptos-bsicos-de-programacin-orientada-a-objetos-poo> pp.22